A shared-memory-parallel implementation of Galerkin-FEM on the Poisson equation with OpenMP in C

High Performance Programming 1TD062

Isak Guardado Sundqvist

August 11, 2026



UPPSALA
UNIVERSITET

Contents

1 Background **1**

2 Problem formulation **2**

3 Theory **4**
3.1 Weak derivatives . . . 4
3.2 Sobolev spaces . . . 5
3.3 Piecewise linear functions . . . 6
3.4 The Galerkin approximation method . . . 7
3.5 Sparse matrices and CSR storage . . . 9
3.6 The Conjugate Gradient method . . . 10
3.7 Shared-memory parallelism and OpenMP . . . 11

4 Implementation **11**
4.1 Application of the Galerkin method on the stated problem . . . 12
4.2 Serial Baseline . . . 15
4.2.1 `main.c` . . . 15
4.2.2 `mesh.c` . . . 16
4.2.3 `csr.c` . . . 16
4.2.4 `FEM.c` . . . 17
4.2.5 `CG.c` . . . 18
4.3 Serial optimization . . . 18
4.3.1 `csr.c` . . . 19
4.3.2 `FEM.c` . . . 19
4.3.3 `CG.c` . . . 20
4.4 Parallelization with OpenMP . . . 21
4.4.1 `csr.c` . . . 21
4.4.2 `FEM.c` . . . 22
4.4.3 `CG.c` . . . 23

5 Performance experiments **23**
5.1 Experimental platform . . . 23
5.2 Serial Experiments . . . 24
5.3 Parallel Experiments . . . 24

6 Results **25**
6.1 FEM solver numerical results . . . 25
6.2 Serial Results . . . 26
6.3 Parallel Results . . . 26
6.3.1 Strong Scaling . . . 26
6.3.2 Weak Scaling . . . 28

7 Discussion **29**
7.1 Serial optimization . . . 29
7.1.1 `csr.c` . . . 30
7.1.2 `CG.c` . . . 30
7.1.3 `FEM.c` . . . 31

7.2 Parallelization 32
7.2.1 `csr.c` 32
7.2.2 `FEM.c` 32
7.2.3 `CG.c` and strong- and weak-scaling behavior 33
7.2.4 Identified parallelization bottlenecks and proposed improvements . . 35

8 Conclusions **36**

A Raw experiment data **38**

1 Background

The numerical solution of partial differential equations (PDEs) constitutes one of the fundamental computational tasks in modern science and engineering. Mathematical models describing physical phenomena such as structural deformation, heat transfer, fluid flow, electromagnetic fields, and diffusion processes are frequently formulated as PDEs. While analytical solutions exist for a limited number of highly idealized problems, realistic engineering applications almost invariably require numerical approximation techniques capable of handling complex geometries, heterogeneous materials, and intricate boundary conditions.

Among the available numerical methods, finite element methods (FEM) have become one of the most widely adopted discretization techniques for initial- and boundary-value problems. Since its development from the Ritz and Galerkin methods during the middle of the twentieth century, FEM has evolved into a standard simulation framework used throughout industry. Today, it forms the computational foundation of commercial multiphysics software packages such as ANSYS (though, computational fluid dynamics overwhelmingly uses finite-volume methods), Abaqus, and COMSOL Multiphysics, where it is routinely applied to the design and verification of products ranging from aircraft structures and automotive components to power transformers, biomedical devices, and civil infrastructure (Liu, Li, and Park 2022).

Although the mathematical formulation of the finite element method is well established, its practical application presents substantial computational challenges. The discretization of realistic engineering domains often produces systems containing millions, in extreme cases perhaps even hundreds of millions, of unknowns. At such scales, one could argue that for plausibility, computational performance becomes as important as numerical accuracy. Memory consumption, data locality, efficient iterative solvers, and exploitation of parallelism in computer hardware could determine whether a simulation completes in seconds, hours, days, or becomes computationally infeasible altogether.

The sparse linear systems arising from finite element discretizations are particularly well suited to iterative Krylov subspace methods, among which the Conjugate Gradient (CG) algorithm remains one of the most widely used solvers for symmetric positive-definite systems (Gutknecht 2007). Though, the computational cost of these methods is typically dominated by repeated sparse matrix–vector multiplications and vector operations. Even with sparse storage, since these linear systems can become extremely large, solvers often become limited by memory bandwidth rather than floating-point throughput (Bell and Garland 2009). This makes memory-conscious data-structure design and generally careful implementation essential for achieving high performance on modern computer architectures.

The continuing increase in model complexity has further elevated the importance of parallel computing. While processor clock frequencies have largely stagnated during the past two decades, computational performance has instead advanced through increasing numbers of processing cores and wider vector units. Consequently, efficient scientific software must increasingly exploit parallelism in order to utilize contemporary hardware effectively. Shared-memory programming models such as OpenMP provide a relatively accessible mechanism for distributing computational work across multiple CPU cores while allowing threads to operate on a common memory space (OpenMP Architecture Review

Board 2024). This programming model is particularly attractive for workstation- and server-scale finite element simulations, where distributed-memory approaches might not be at all available, or where the associated additional complexity and communication overhead isn't yet justified.

The objective of this project is therefore not merely to implement a finite element solver, but to investigate how algorithmic design, memory-efficient data structures, serial optimization techniques, and shared-memory parallelization collectively influence computational performance. Beginning from a straightforward baseline implementation, optimization efforts are progressively carried out through attempted improvements in sparse matrix construction, finite element assembly, and iterative solution procedures before being parallelized using OpenMP. The resulting implementation is subsequently evaluated through serial, strong-scaling, and weak-scaling experiments in order to quantify both the benefits and the practical limitations of the applied optimization strategies.

2 Problem formulation

This project considers the numerical solution of the Poisson equation on a two-dimensional domain. The chosen computational domain is the open unit disk,

$$\Omega = \left\{(x, y) \in \mathbb{R}^2 : x^2 + y^2 < 1\right\}, \tag{1}$$

with boundary

$$\partial\Omega = \left\{(x, y) \in \mathbb{R}^2 : x^2 + y^2 = 1\right\}. \tag{2}$$

The boundary-value problem is to find a scalar function $u : \overline{\Omega} \to \mathbb{R}$ satisfying

$$-\Delta u = 1 \qquad \text{in } \Omega, \tag{3}$$

together with the homogeneous Dirichlet boundary condition

$$u = 0 \qquad \text{on } \partial\Omega. \tag{4}$$

Here, Δ denotes the two-dimensional Laplace operator

$$\Delta u = \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2}. \tag{5}$$

The right-hand side of equation 3 represents a spatially uniform source term. Depending on the physical interpretation, the same mathematical problem may describe, for example, steady-state heat conduction with uniform internal heat generation, diffusion with a constant source, or an electrostatic potential generated by a uniform charge distribution.

The problem above is stated in its strong form. In this formulation, the differential equation is required to hold pointwise continuously throughout the domain. A classical

solution must therefore possess sufficiently regular second derivatives for the Laplace operator to be defined in the ordinary sense. This requirement is natural when an analytical solution is sought, but it is not directly compatible with the piecewise linear approximation functions used in the Galerkin finite element method. Such piecewise linear functions are continuous across element boundaries, but their gradients are generally discontinuous between adjacent elements (they form sharp angles, or "kinks"). Consequently, classical second derivatives are not defined globally, since there isn't any "real curvature" in the function in the continuous sense – the function is just a concatenation of linear segments.

The continuous problem is also infinite-dimensional: the unknown is a function with an infinite amount of points at which it can be evaluated, rather than a finite collection of scalar values. Except for comparatively simple combinations of geometry, source terms, coefficients, and boundary conditions – exactly like our problem – a closed-form analytical solution cannot generally be obtained. Industrial finite element models may additionally contain exceptionally irregular geometries, spatially varying material properties, multiple physical regions, and complex boundary conditions. A numerical method must therefore replace the continuous problem with a finite-dimensional model and approximation that can be represented and solved on a computer.

For the particular model problem considered here, an analytical solution is available:

$$u(x, y) = \frac{1 - x^2 - y^2}{4}. \tag{6}$$

Its second derivatives are

$$\frac{\partial^2 u}{\partial x^2} = -\frac{1}{2}, \qquad \frac{\partial^2 u}{\partial y^2} = -\frac{1}{2}, \tag{7}$$

and therefore

$$-\Delta u = -\left(-\frac{1}{2} - \frac{1}{2}\right) = 1. \tag{8}$$

Furthermore, $u = 0$ whenever $x^2 + y^2 = 1$, so the boundary condition is also satisfied. This analytical solution is not used when computing the numerical solution. Instead, it provides a reference against which the correctness and discretization error of the finite element approximation can be evaluated.

The computational objective is to construct an approximation u_h of u on a triangular mesh of Ω. The approximation is represented using continuous functions that are linear (planes) within each triangle. The continuous differential equation is reformulated as a weak problem, reducing the required derivative order and making it compatible with the approximation space, which has defined weak first derivatives. Restricting the weak problem to a finite-dimensional geometric vector space then produces a sparse linear system of the form

$$A\boldsymbol{\xi} = \boldsymbol{b}, \tag{9}$$

where A is the finite element stiffness matrix, $\boldsymbol{b}$ is the assembled load vector, and $\boldsymbol{\xi}$ contains the unknown nodal values of the approximate solution.

The purpose of the project extends beyond obtaining a numerical approximation of this particular Poisson problem. The unit disk and constant source term provide a controlled model problem with a known exact solution, allowing numerical correctness to be separated from implementation performance. The complete computational pipeline includes mesh parsing, Compressed-Row Sparse (CSR) matrix pattern construction, finite element stiffness matrix and load vector assembly, enforcement of boundary conditions, and iterative CG solution of the resulting linear system. The implementation is first constructed in a straightforward serial form, then serially optimized, and finally parallelized for shared-memory using OpenMP.

The principal objective is to investigate how choices involving algorithms, memory access patterns, synchronization, and load distribution affect the performance and scalability of a finite element solver. Although the model problem is mathematically simple, the main computational operations are representative of those encountered in substantially larger scientific and industrial finite element applications.

3 Theory

In this section, the mathematical and technical principles that underpin the project will be presented.

3.1 Weak derivatives

The classical derivative of a function is defined continuously pointwise and therefore requires the function to be sufficiently regular in a neighborhood of each point. This requirement is too restrictive for many numerical approximations. In particular, a continuous piecewise linear finite-element function is differentiable within each mesh element, but its gradient may change discontinuously across the interface between two adjacent elements. Its classical derivative is consequently not defined at every point of the domain, and therefore a classical global derivative of such a function is not generally well-defined.

The concept of a weak derivative extends differentiation to functions that need not be classically differentiable everywhere, and its definition is based on integration by parts. Let u be continuously differentiable on an open interval $\Omega \subset \mathbb{R}$, and let $v \in C_c^\infty(\Omega)$ be a smooth test function with compact support in Ω (note that this extends to functions v that simply vanish on the actual boundary of Ω, indeed this is the case for our model). Since v vanishes near the boundary of the interval, integration by parts gives

$$\int_\Omega u'(x)v(x)dx = -\int_\Omega u(x)v'(x)dx. \tag{10}$$

The expression on the right remains meaningful even when the classical derivative u' does not exist. This observation is used to define differentiation through an integral identity rather than through pointwise limits, i.e classical differentiability.

A locally integrable function w is called the weak derivative of u if

$$\int_\Omega w(x)v(x)dx = -\int_\Omega u(x)v'(x)dx \tag{11}$$

for every test function $v \in C_c^\infty(\Omega)$. When it exists, the weak derivative is unique up to sets of measure zero. If u is also classically differentiable, its weak derivative agrees almost everywhere with its classical derivative (Chen n.d.; Viaclovsky 2004; Wikipedia contributors n.d.).

The finite-element functions used in this project are continuous and linear within each triangle. Their classical gradients are constant in the interior of each element but generally jump across element boundaries. These boundaries have zero area in the two-dimensional domain, so the gradient is defined almost everywhere. Moreover, the element-wise gradient is square-integrable and coincides almost everywhere with the weak gradient. Continuous piecewise linear finite-element functions therefore possess weak first derivatives even though their classical first derivatives are not defined globally at every point.

This distinction is central to the finite element method. It permits the approximate solution to be constructed from continuous piecewise linear functions while still allowing first derivatives to appear in the weak formulation (also called "variational" formulation).

3.2 Sobolev spaces

Sobolev spaces collect functions according to the integrability of the functions themselves and of their weak derivatives. These spaces provide the natural mathematical setting for weak formulations of partial differential equations and underpin finite-element methods.

The space $L^2(\Omega)$ consists of functions whose squared magnitude is integrable over the domain Ω:

$$L^2(\Omega) = \left\{ v : \int_\Omega |v|^2 d\boldsymbol{x} < \infty \right\}, \tag{12}$$

with corresponding norm

$$\|v\|_{L^2(\Omega)} = \left(\int_\Omega |v|^2 d\boldsymbol{x} \right)^{1/2} \tag{13}$$

Functions in $L^2(\Omega)$ are allowed to have limited pointwise regularity, provided that their squared magnitude remains integrable over the domain. Functions that differ only on a set of measure zero have the same integrals, and as such, are regarded as equivalent within $L^2(\Omega)$.

The first-order Sobolev space $H^1(\Omega)$ consists of functions that belong to $L^2(\Omega)$ and whose first weak partial derivatives also belong to $L^2(\Omega)$. In two spatial dimensions, it is defined as

$$H^1(\Omega) = \left\{ v \in L^2(\Omega) : D_x v \in L^2(\Omega), \quad D_y v \in L^2(\Omega) \right\}, \tag{14}$$

where $D_x v$ and $D_y v$ are defined as the weak first partial derivatives of v.

The H^1-norm measures both the magnitude of the function and the magnitude of its weak gradient in the L^2 sense:

$$\|v\|_{H^1(\Omega)} = \left(\|v\|^2_{L^2(\Omega)} + \|D_x v\|^2_{L^2(\Omega)} + \|D_y v\|^2_{L^2(\Omega)}\right)^{1/2}, \tag{15}$$

which can be equivalently written with the weak gradient

$$\|v\|_{H^1(\Omega)} = \left(\|v\|^2_{L^2(\Omega)} + \|\nabla v\|^2_{L^2(\Omega)}\right)^{1/2}. \tag{16}$$

The homogeneous Dirichlet boundary condition is represented using the subspace $H^1_0(\Omega)$. This space consists of functions in $H^1(\Omega)$ *that also vanish on the boundary*:

$$H^1_0(\Omega) = \left\{v \in H^1(\Omega) : v = 0 \text{ on } \partial\Omega\right\}. \tag{17}$$

The distinction between $H^1(\Omega)$ and $H^1_0(\Omega)$ is important for the model problem. The solution and test functions must possess square-integrable weak first derivatives, so that expressions involving their gradients are well defined. They must also satisfy the homogeneous Dirichlet condition. The appropriate function space is therefore $H^1_0(\Omega)$.

Continuous piecewise linear finite-element functions belong to $H^1(\Omega)$. Their gradients are constant inside each triangle and may jump across element interfaces, but these jumps do not prevent the weak gradient from being square-integrable. When the functions are constrained to vanish on the boundary, they belong to $H^1_0(\Omega)$. This makes continuous piecewise linear functions suitable as both trial and test functions in the finite-element approximation of the model problem (Chen n.d.; Viaclovsky 2004; Wikipedia contributors 2026b).

3.3 Piecewise linear functions

A function does not need to be described by a single expression over an entire domain. It may instead be defined separately on a collection of smaller subdomains. Such a function is called piecewise defined. In this project, the domain is divided into non-overlapping triangles – making it a so-called mesh – and the functions of interest are linear within each triangle.

Let $\mathcal{T}_h$ denote a triangulation of the domain Ω, consisting of non-overlapping triangles K. The space of continuous piecewise linear functions over this triangulation is defined as

$$V_h := \left\{v_h \in C^0(\overline{\Omega}) : v_h|_K \in \mathbb{P}_1(K) \quad \forall K \in \mathcal{T}_h\right\}, \tag{18}$$

where $C^0(\overline{\Omega})$ denotes the space of continuous functions on the closure of Ω, and $\mathbb{P}_1(K)$ denotes the space of linear polynomials on the triangle K. Since the functions in V_h are continuous and possess square-integrable weak first derivatives, the space satisfies

$$V_h \subset H^1(\Omega). \tag{19}$$

A function $u_h \in V_h$ is therefore linear within each triangle. Its restriction to a triangle K can be written as

$$u_h|_K(x, y) = a_K + b_K x + c_K y, \qquad K \in \mathcal{T}_h, \tag{20}$$

making it obvious that the function is a plane on each triangle.

A continuous piecewise linear function is completely determined by its values at the vertices, or nodes, of the triangulation. Suppose the triangulation has N vertices, denoted by $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N$. For every vertex $\boldsymbol{x}_i$, a nodal basis function φ_i is defined by

$$\varphi_i(\boldsymbol{x}_j) = \begin{cases} 1, & i = j, \\ 0, & i \neq j. \end{cases} \tag{21}$$

Each basis function, also commonly called tent function (for the $\mathbb{P}_1$ case), is continuous over the domain and linear within every triangle. It is nonzero only on triangles that contain its associated vertex. This region is commonly called the support of the basis function.

Any continuous piecewise linear function v_h can be expressed as a linear combination of the nodal basis functions:

$$v_h(\boldsymbol{x}) = \sum_{i=1}^{N} \xi_i \varphi_i(\boldsymbol{x}), \tag{22}$$

where $\xi_i = v_h(\boldsymbol{x}_i)$ is the evaluation of the function at vertex $\boldsymbol{x}_i$. The continuous function is therefore represented by the finite collection of nodal values $\xi_1, \ldots, \xi_N$, in this way.

The nodal basis functions also satisfy the partition-of-unity property

$$\sum_{i=1}^{N} \varphi_i(\boldsymbol{x}) = 1 \tag{23}$$

at every point covered by the triangulation (Wikipedia contributors 2026b). These properties make continuous piecewise linear functions convenient for representing spatially varying quantities over triangulated domains. Their role in constructing a finite-dimensional approximation of the model problem is introduced in the following section.

3.4 The Galerkin approximation method

After a differential equation has been transformed from its strong form to a weak formulation, the problem is generally posed in an infinite-dimensional function space V. The weak problem can be written in the abstract form: find $u \in V$ such that

$$a(u, v) = L(v) \qquad \forall v \in V, \tag{24}$$

where $a(u, v)$ is a bilinear form containing the terms involving the unknown solution, while $L(v)$ is a linear functional containing the known data of the problem. Their specific forms of course depend on the differential equation being considered.

Since the space V is infinite-dimensional, the solution cannot be represented directly using a finite number of values. The Galerkin method therefore replaces V with a finite-dimensional subspace $V_h \subset V$, which generally is chosen to be a space of piecewwise defined functions over the partitioned original domain. For example, V is often H_0^1 and V_h could then be the space of piecewise linear polynomials, as in equation 18.

When imposing homogeneous Dirichlet boundary conditions, the functions in V_h are additionally constrained to vanish on the boundary, so that $V_h \subset H_0^1(\Omega)$.

The Galerkin approximation consists of finding $u_h \in V_h$ such that

$$a(u_h, v_h) = L(v_h) \qquad \forall v_h \in V_h. \tag{25}$$

The same finite-dimensional space is therefore used for both the approximate solution, called the trial function, and the test functions. This is the defining feature of the Galerkin method.

Let $\varphi_1, \ldots, \varphi_N$ be a basis for V_h. The approximate solution can then be spectrally represented as

$$u_h(\boldsymbol{x}) = \sum_{j=1}^{N} \xi_j \varphi_j(\boldsymbol{x}), \tag{26}$$

where the coefficients $\xi_1, \ldots, \xi_N$ are unknown. For the nodal basis functions introduced previously, these coefficients correspond to the values of u_h at the mesh vertices. Since every function in V_h is a linear combination of the basis functions in this way, it is sufficient to require the Galerkin equation 25 to hold for each individual basis function. Setting the test function $v_h = \varphi_i$ gives

$$a(u_h, \varphi_i) = L(\varphi_i), \qquad i = 1, \ldots, N. \tag{27}$$

Substituting the basis expansion of u_h yields (because of the bilinearity of $a(\cdot, \cdot)$)

$$\sum_{j=1}^{N} \xi_j a(\varphi_j, \varphi_i) = L(\varphi_i), \qquad i = 1, \ldots, N. \tag{28}$$

Each $a(\varphi_j, \varphi_i)$ is defined as a matrix entry

$$A_{ij} = a(\varphi_j, \varphi_i), \tag{29}$$

and each $L(\varphi_i)$ is defined as a component of the so-called load vector

$$b_i = L(\varphi_i). \tag{30}$$

The Galerkin approximation can consequently be written as the finite system of linear equations

$$A\boldsymbol{\xi} = \boldsymbol{b}, \tag{31}$$

where $\boldsymbol{\xi}$ contains the unknown nodal values of the approximate solution. The original problem of finding an unknown function in an infinite-dimensional space has thereby been transformed into the problem of determining a finite vector of nodal values.

If u denotes the exact weak solution, subtracting the continuous and discrete formulations gives

$$a(u - u_h, v_h) = 0 \qquad \forall v_h \in V_h. \tag{32}$$

This property is known as Galerkin orthogonality. It states that the approximation error is orthogonal to the discrete function space with respect to the bilinear form $a(\cdot, \cdot)$ - that is, u_h is the approximation in V_h that produces an error that projects the smallest amount possible (not at all) unto V_h (Wikipedia contributors 2026c; Wikipedia contributors 2026b). The particular bilinear form, linear functional, and resulting matrix entries for the model problem in this project will be derived when the mathematical implementation is presented.

3.5 Sparse matrices and CSR storage

The Galerkin method produces a linear system of the form

$$A\boldsymbol{\xi} = \boldsymbol{b}. \tag{33}$$

For piecewise linear functions, each basis function is nonzero only on a small group of triangles surrounding its associated mesh vertex. Consequently, two basis functions interact only if their supports overlap. Most entries of the matrix $\boldsymbol{A}$ are therefore zero, making it a sparse matrix.

Storing every entry of a sparse matrix in a conventional two-dimensional array would waste considerable (indeed, sometimes unfathomable) memory just storing zeroes. The Compressed Sparse Row format, abbreviated CSR, avoids this by storing only the nonzero entries of the matrix. A matrix in CSR format is represented using three arrays:

- `values`, containing the nonzero matrix entries stacked row-major,
- `col_idx`, containing the column index of each stored entry,
- `row_ptr`, indicating where each matrix row begins in the other two arrays.

If row i begins at position `row_ptr[i]` and ends immediately before `row_ptr[i+1]`, then its contribution in the matrix-vector product $A\boldsymbol{x}$ is calculated as

$$(A\boldsymbol{x})_i = \sum_{k=\texttt{row_ptr}[i]}^{\texttt{row_ptr}[i+1]-1} \texttt{values}[k] x_{\texttt{col_idx}[k]}. \tag{34}$$

CSR storage substantially reduces the memory required for large sparse systems and permits the nonzero entries of each row to be traversed consecutively. It is therefore well suited to the sparse matrices produced by the finite element method and to the repeated matrix-vector products required by iterative linear solvers.

3.6 The Conjugate Gradient method

The Conjugate Gradient method, commonly abbreviated CG, is an iterative method for solving linear systems

$$A\boldsymbol{x} = \boldsymbol{b}, \tag{35}$$

where A is symmetric and positive definite. These properties are satisfied by the stiffness matrix obtained from the finite-element discretization of the model problem after the homogeneous Dirichlet boundary conditions have been imposed.

Starting from an initial approximation $\boldsymbol{x}_0$, the residual is defined as

$$\boldsymbol{r}_0 = \boldsymbol{b} - A\boldsymbol{x}_0. \tag{36}$$

The initial search direction is chosen as

$$\boldsymbol{p}_0 = \boldsymbol{r}_0. \tag{37}$$

At iteration k, the step length is calculated as

$$\alpha_k = \frac{\boldsymbol{r}_k^T \boldsymbol{r}_k}{\boldsymbol{p}_k^T A \boldsymbol{p}_k}. \tag{38}$$

The approximate solution and residual are then updated according to

$$\boldsymbol{x}_{k+1} = \boldsymbol{x}_k + \alpha_k \boldsymbol{p}_k \quad \text{and} \quad \boldsymbol{r}_{k+1} = \boldsymbol{r}_k - \alpha_k A \boldsymbol{p}_k. \tag{39}$$

The next search direction is formed as

$$\boldsymbol{p}_{k+1} = \boldsymbol{r}_{k+1} + \beta_k \boldsymbol{p}_k, \tag{40}$$

where

$$\beta_k = \frac{\boldsymbol{r}_{k+1}^T \boldsymbol{r}_{k+1}}{\boldsymbol{r}_k^T \boldsymbol{r}_k}. \tag{41}$$

The search directions are conjugate with respect to A, meaning that

$$\boldsymbol{p}_i^T A \boldsymbol{p}_j = 0, \qquad i \neq j. \tag{42}$$

This prevents successive iterations from repeatedly correcting the solution in the same direction. In exact arithmetic, CG produces the exact solution after at most N iterations for an $N \times N$ system, although in practice it is stopped earlier when the residual is sufficiently small.

A common stopping condition is based on the relative residual,

$$\frac{\|\boldsymbol{r}_k\|_2}{\|\boldsymbol{b}\|_2} < \varepsilon, \tag{43}$$

where ε is a prescribed tolerance (Wikipedia contributors 2026a).

CG is well suited to large sparse systems because it does not require a direct matrix factorization. Each iteration mainly consists of one sparse matrix-vector multiplication, several vector updates, and inner products. The sparse matrix-vector multiplication is generally the most computationally expensive operation in the algorithm and therefore has a strong influence on the performance of the solver.

3.7 Shared-memory parallelism and OpenMP

In shared-memory parallel programming, several threads execute concurrently while accessing the same address space. This allows the threads to operate on common arrays and data structures without explicitly transferring data between them.

OpenMP provides compiler directives and library functions for shared-memory parallel programming in C. A parallel region creates a team of threads that executes the enclosed code concurrently. Work-sharing directives can then divide work among the threads. In particular, the `omp for` directive distributes the iterations of a loop, provided that the iterations are independent.

Variables used inside a parallel region may be either shared or private. Shared variables refer to the same memory location for all threads, whereas each thread receives its own copy of a private variable. Incorrect simultaneous updates to shared data may cause data races and produce unpredictable results; some parallel computations, such as inner products, require the partial results from all threads to be combined. OpenMP provides reduction operations for this purpose. Each thread computes a private partial value, after which the values are combined into one result.

Parallel execution also introduces overhead. Threads may need to wait at synchronization barriers, and unevenly divided work can cause load imbalance. Performance may additionally be limited by cache misses, memory bandwidth, false sharing, and non-uniform memory access on a multi-socket system. Consequently, increasing the number of threads does not necessarily produce a proportional reduction in runtime (OpenMP Architecture Review Board 2024).

4 Implementation

In this section, the implementation of the solution for the stated problem will be presented. This will include the mathematical modeling of the problem and solution method, as well

as the practical computational implementation in C; especially the motivations behind design choices, optimizations, and parallelization strategies.

4.1 Application of the Galerkin method on the stated problem

The general Galerkin framework was introduced in Section 3.4. For the model problem considered here, the method is obtained by multiplying the Poisson equation by an arbitrary test function $v \in H_0^1(\Omega)$ and integrating over the domain:

$$\int_\Omega (-\Delta u) v \mathrm{d}\boldsymbol{x} = \int_\Omega v \mathrm{d}\boldsymbol{x}. \tag{44}$$

Applying Green's identity (equivalent to integration by parts in 2D) gives

$$\int_\Omega \nabla u \cdot \nabla v \mathrm{d}\boldsymbol{x} - \int_{\partial\Omega} v \frac{\partial u}{\partial n} \mathrm{d}s = \int_\Omega v \mathrm{d}\boldsymbol{x}. \tag{45}$$

The boundary integral vanishes because the trace of every test function in $H_0^1(\Omega)$ is zero on $\partial\Omega$. The weak problem is therefore to find $u \in H_0^1(\Omega)$ such that

$$a(u, v) = L(v) \qquad \forall v \in H_0^1(\Omega), \tag{46}$$

where

$$a(u, v) = \int_\Omega \nabla u \cdot \nabla v \mathrm{d}\boldsymbol{x}, \qquad L(v) = \int_\Omega v \mathrm{d}\boldsymbol{x}. \tag{47}$$

The disk is discretized using a triangular mesh $\mathcal{T}_h$. Since the mesh consists of straight-sided triangles, their union forms a polygonal approximation Ω_h of the circular domain. First, define the unconstrained finite-element space

$$\widetilde{V}_h = \left\{ v_h \in C^0(\overline{\Omega_h}) : v_h|_K \in \mathbb{P}_1(K) \quad \forall K \in \mathcal{T}_h \right\}. \tag{48}$$

The homogeneous Dirichlet condition is incorporated through the subspace

$$V_h = \left\{ v_h \in \widetilde{V}_h : v_h = 0 \text{ on } \partial\Omega_h \right\}. \tag{49}$$

The discrete Galerkin problem is consequently to find $u_h \in V_h$ such that

$$\int_{\Omega_h} \nabla u_h \cdot \nabla v_h \mathrm{d}\boldsymbol{x} = \int_{\Omega_h} v_h \mathrm{d}\boldsymbol{x} \qquad \forall v_h \in V_h. \tag{50}$$

Let $\varphi_1, \ldots, \varphi_{N_I}$ denote the nodal basis functions of V_h, associated with the N_I interior mesh vertices. Since these functions span V_h, the finite-element solution can be written as

$$u_h(\boldsymbol{x}) = \sum_{j=1}^{N_I} \xi_j \varphi_j(\boldsymbol{x}). \tag{51}$$

Substituting this expansion into the discrete weak problem gives

$$\int_{\Omega_h} \nabla \left(\sum_{j=1}^{N_I} \xi_j \varphi_j \right) \cdot \nabla v_h \mathrm{d}\boldsymbol{x} = \int_{\Omega_h} v_h \mathrm{d}\boldsymbol{x} \qquad \forall v_h \in V_h. \tag{52}$$

By the linearity of the gradient and the integral, this becomes

$$\sum_{j=1}^{N_I} \xi_j \int_{\Omega_h} \nabla \varphi_j \cdot \nabla v_h \mathrm{d}\boldsymbol{x} = \int_{\Omega_h} v_h \mathrm{d}\boldsymbol{x} \qquad \forall v_h \in V_h. \tag{53}$$

Since $\varphi_1, \ldots, \varphi_{N_I}$ form a basis of V_h, it is sufficient to enforce equation 53 for each basis function. Choosing $v_h = \varphi_i$ therefore gives

$$\sum_{j=1}^{N_I} \xi_j \int_{\Omega_h} \nabla \varphi_j \cdot \nabla \varphi_i \mathrm{d}\boldsymbol{x} = \int_{\Omega_h} \varphi_i \mathrm{d}\boldsymbol{x}, \qquad i = 1, \ldots, N_I. \tag{54}$$

Defining

$$A_{ij} = \int_{\Omega_h} \nabla \varphi_j \cdot \nabla \varphi_i \mathrm{d}\boldsymbol{x}, \qquad b_i = \int_{\Omega_h} \varphi_i \mathrm{d}\boldsymbol{x}, \tag{55}$$

the N_I equations can be collected into the algebraic system

$$A\boldsymbol{\xi} = \boldsymbol{b}, \tag{56}$$

where $A \in \mathbb{R}^{N_I \times N_I}$, $\boldsymbol{\xi} \in \mathbb{R}^{N_I}$, and $\boldsymbol{b} \in \mathbb{R}^{N_I}$.

The domain Ω_h is the union of the triangles in $\mathcal{T}_h$. The integrals defining the matrix and load vector can therefore be decomposed into sums of integrals over the elements:

$$A_{ij} = \sum_{K \in \mathcal{T}_h} \int_K \nabla \varphi_j \cdot \nabla \varphi_i \mathrm{d}\boldsymbol{x}, \qquad b_i = \sum_{K \in \mathcal{T}_h} \int_K \varphi_i \mathrm{d}\boldsymbol{x}. \tag{57}$$

A basis function is nonzero only on the elements sharing its associated vertex. Consequently, an element contributes to A_{ij} only if both φ_i and φ_j are supported on that element. This local support is what gives the global stiffness matrix its sparse structure – most of the integrals come out to zero.

Consider a triangular element K. Its three vertices are numbered locally as

$$\begin{bmatrix}(x_1, y_1)\\(x_2, y_2)\\(x_3, y_3)\end{bmatrix}. \tag{58}$$

The signed determinant D_K and area $|K|$ of the element are

$$D_K = (x_2 - x_1)(y_3 - y_1) - (x_3 - x_1)(y_2 - y_1), \qquad |K| = \frac{|D_K|}{2}. \tag{59}$$

Let $\lambda_1, \lambda_2, \lambda_3$ denote the local linear nodal basis functions restricted to K. Since they are just planes, their gradients are constant within the element, and hence can be written as

$$\begin{bmatrix}\nabla\lambda_1 & \nabla\lambda_2 & \nabla\lambda_3\end{bmatrix} = \frac{1}{D_K} G_K, \tag{60}$$

where

$$G_K = \begin{bmatrix}y_2 - y_3 & y_3 - y_1 & y_1 - y_2\\ x_3 - x_2 & x_1 - x_3 & x_2 - x_1\end{bmatrix}. \tag{61}$$

The local stiffness matrix will then come out to

$$A^{(K)} = \frac{1}{4|K|} G_K^T G_K, \tag{62}$$

with entries

$$A_{ij}^{(K)} = \int_K \nabla\lambda_j \cdot \nabla\lambda_i \, \mathrm{d}\boldsymbol{x}, \qquad i, j \in \{1, 2, 3\}. \tag{63}$$

For the constant source term, the local load vector is

$$\boldsymbol{b}^{(K)} = \frac{|K|}{3} \begin{bmatrix}1\\1\\1\end{bmatrix} \tag{64}$$

Let I_1, I_2, I_3 denote the global node indices corresponding to the three vertices of K. The local contributions are added to the global system through

$$A_{I_i I_j} \leftarrow A_{I_i I_j} + A_{ij}^{(K)}, \qquad i, j \in \{1, 2, 3\}, \tag{65}$$

$$b_{I_i} \leftarrow b_{I_i} + b_i^{(K)}, \qquad i \in \{1, 2, 3\}. \tag{66}$$

A formulation based directly on V_h contains only the N_I interior unknowns. The actual implementation in this project instead assembles all N mesh nodes so that the original node numbering can be retained. This produces an $N \times N$ system before the boundary conditions are imposed.

For each boundary node p, the homogeneous Dirichlet condition $\xi_p = 0$ is imposed manually by setting

$$A_{pp} = 1, \qquad A_{pj} = A_{jp} = 0 \quad \text{for } j \neq p, \qquad b_p = 0. \tag{67}$$

Zeroing both the boundary rows and columns preserves the symmetry of the matrix, and the interior part of the resulting system is identical to the reduced system obtained directly from V_h. The final matrix is therefore symmetric positive definite and can be solved using the Conjugate Gradient method.

4.2 Serial Baseline

A serial baseline was first developed to provide a correct reference implementation for the later serial optimizations and OpenMP parallelization. This was the first working complete program build, and it was coded from start to end not necessarily with a focus on the efficiency of the code; rather, the priority here was to write the code in a way that was the most understandable and readable, at least for the author.

The program followed the complete solution procedure from mesh input to verification and consisted of the following modules:

- `main.c`: program entry point, control, timing, and verification,
- `mesh.c`: mesh input and storage,
- `csr.c`: construction of the sparse-matrix structure,
- `FEM.c`: finite-element assembly of the stiffness matrix and load vector, as well as application of boundary conditions,
- `CG.c`: solution of the linear system.

All stages of the baseline were executed serially. The implementation closely followed the mathematical algorithms presented in the preceding sections, with a deliberately modular approach to the functions and loops used for the main computational operations.

Notice that all versions of the implementation discussed in this report used the compiler optimization flags `-O3` and `-march=native`.

4.2.1 `main.c`

The `main.c` module controlled the computational pipeline. The principal program objects created in `main()` were a `Mesh` structure, a `CSRMatrix` structure, and the dynamically allocated arrays `f` and `x`, which stored the load vector and solution vector, respectively. The program accepted a Gmsh mesh file as input and called the main functions of the remaining modules in the following order: `read_mesh()`, `build_csr_pattern()`, `sparse_global_assemble()`, `apply_dirichlet_bc()`, and `conjugate_gradient()`. The solution vector was initialized to zero, which also provided the initial approximation for CG (the zero vector is a very commonly used first guess used in CG).

The runtime of each main stage was measured separately using `clock_gettime()` with the monotonic system clock. This made it possible to distinguish the time spent on mesh input, CSR construction, assembly, boundary conditions, and the linear solver,

respectively and collectively. Verification and optional output were performed after the timed solver pipeline so that they did not affect the performance measurements.

Correctness was checked by using `csr_matvec()` to independently recompute $A\boldsymbol{\xi}$ and then evaluate the relative residual

$$\frac{\|\boldsymbol{b} - A\boldsymbol{\xi}\|_2}{\|\boldsymbol{b}\|_2}. \tag{68}$$

The largest absolute solution value on the boundary was also recorded to verify the homogeneous Dirichlet condition. Finally, the nodal solution was compared with the analytical solution

$$u(x, y) = \frac{1 - x^2 - y^2}{4}. \tag{69}$$

When requested, the function `export_solution()` exported the node coordinates, solution values, and element connectivity to CSV files for visualization.

4.2.2 `mesh.c`

The unit-disk geometry was defined in the Gmsh mesh configuration file `disk.geo` and discretized with straight-sided triangular elements using Gmsh. The resulting mesh was stored in the Gmsh 2.2 ASCII format and read by the function `read_mesh()` in `mesh.c`.

The central data structure of the module was `Mesh`, which stored the numbers of nodes and elements together with the arrays `nodes`, `elements`, and `is_boundary`. Each entry of `nodes` was a `Node` structure containing the two coordinates `x` and `y`, while each entry of `elements` was an `Element` structure containing the three vertex indices `n1`, `n2`, and `n3`. The boundary information was represented by the `unsigned char` array `is_boundary`, with an entry equal to 1 for a boundary node and 0 otherwise.

The parser processed the `$Nodes` and `$Elements` sections of the mesh file, while Gmsh element tags and element types that were not required by the solver were skipped. Triangular elements, identified as Gmsh element type 2, were stored using the indices of their three vertices, whereas the global mesh-boundary line segments were identified as element type 1 and both endpoints (i.e nodes) of every segment were flagged in `is_boundary`. Since Gmsh uses one-based node numbering while C arrays use zero-based indexing, every node index was reduced by one during input.

4.2.3 `csr.c`

The stiffness matrix was represented by the `CSRMatrix` structure and stored in the CSR format described in Section 3.5. This structure contained the matrix dimension `n`, the number of stored entries `nnz`, and the three CSR arrays `row_ptr`, `col_idx`, and `val`. Since the sparsity pattern depends only on the mesh connectivity, it was constructed by `build_csr_pattern()` before the finite-element numerical assembly.

In a triangle, each of its three nodes interacts with all three nodes of the same element in the sense that they are connected by edges. Each stiffness matrix row corresponds to a node: wherever there is a nonzero entry, the node corresponding to that column index

is connected to the row node. The baseline construction represented these interactions using one dynamically growing adjacency array for every matrix row: `adj` was an array of pointers for which `adj[i]` pointed to the column indices currently associated with row i, while the arrays `adj_size` and `adj_cap` stored the current length and allocated capacity of every row, respectively. For each triangle, its three node indices were appended to the adjacency array of each of its nodes, which included the diagonal connections and produced duplicate entries wherever several triangles connected the same pair of nodes.

The helper function `append()` performed this dynamic insertion. Whenever an adjacency array was full, its initial capacity was set to eight entries or its existing capacity was doubled, after which `realloc()` was used to enlarge the array before the new column index was inserted. This dynamic-append approach was straightforward to implement because no row lengths had to be known beforehand, although it required separately allocated row arrays and potentially (pretty much guaranteed with a huge mesh) several reallocations during construction, which made it a good target for serial optimization.

Each completed adjacency array was sorted using `qsort()`. Repeated column indices then became adjacent and were removed by a single in-place traversal, after which the remaining indices represented the distinct nonzero positions in the corresponding matrix row.

The row lengths were summed to obtain `nnz`, and their prefix sum formed `row_ptr`, while the sorted adjacency arrays were copied consecutively into `col_idx`. The corresponding `val` array was allocated and initialized to zero. Sorting the column indices was also necessary for the assembly stage, since it allowed individual matrix entries to be located by binary search.

4.2.4 `FEM.c`

The `FEM.c` module assembled the stiffness matrix and load vector through the main assembly function `sparse_global_assemble()`. Before assembly, the load-vector array `f` was initialized to zero, after which the elements were processed one at a time.

For each triangle, the coordinates of its three vertices were retrieved from the `nodes` array using the indices stored in its `Element` structure. Here in the baseline, separate functions `local_stiffness()` and `local_load()` were used to calculate the local stiffness matrix `Ke[3][3]` and local load vector `fe[3]`. The first function evaluated the element area and the gradient coefficients required for the nine stiffness entries (see section 4.1), while the second evaluated the area again and assigned the value $|K|/3$ to each of the three local load entries. This separation followed the mathematical formulation directly and made the baseline easy to follow, although it also repeated the triangle area calculation. This made it a target, albeit a small one, for optimization.

The local contributions were inserted using the element connectivity. For every local pair i, j, the corresponding global indices I, J were obtained, after which the helper function `find_position()` performed a binary search over the section of `col_idx` belonging to row I. The returned index identified the position of A_{IJ} in `val`, where the local contribution was accumulated using `A->val[k] += Ke[i][j]`. This search was repeated for all nine entries of every local stiffness matrix, while the three local load contributions were added directly to the global array through `f[I] += fe[i]`.

After assembly, `apply_dirichlet_bc()` imposed the boundary conditions in two sequential passes. The first pass traversed all stored matrix entries and set an entry to zero whenever its row or column belonged to a node flagged in `is_boundary`, while the second pass set the diagonal entry of each boundary row to one and the corresponding load-vector entry to zero. Modifying both rows and columns preserved matrix symmetry and produced the symmetric positive-definite system required by CG. The two-pass approach was an area of improvement.

4.2.5 `CG.c`

The `CG.c` module implemented the standard non-preconditioned Conjugate Gradient algorithm described in section 3.6, with the complete solution process contained in `conjugate_gradient()`. CG was selected because the finite-element matrix is sparse, symmetric, and positive definite after the boundary conditions have been imposed; also, unlike a direct solver, it does not require matrix factorization and instead operates directly on the `CSRMatrix` representation. This makes it a simple and straightforward algorithm to use for this problem.

Three dynamically allocated working vectors were created once at the beginning of the solve: the residual `r`, the search direction `p`, and the matrix–vector product `Ap`. Starting from the zero initial guess stored in `x`, the initial residual and search direction were both equal to the load-vector array `f`.

Each iteration performed the following operations in separate loops:

1. compute $A\boldsymbol{p}$ using `csr_matvec()`, which traversed the CSR matrix row by row,
2. compute the inner product $\boldsymbol{p}^T A\boldsymbol{p}$ using `dot()`
3. update the solution and residual vectors,
4. compute the new residual norm,
5. update the search direction.

The function `csr_matvec()` visited every stored matrix entry once, while the remaining operations traversed vectors of length N, making the matrix–vector multiplication the principal sparse kernel in each iteration.

The solver stopped when the relative residual fell below 10^{-10} or when the limit of 10 000 iterations was reached, and returned both the iteration count and the final relative residual to `main.c`. The quantity $\boldsymbol{p}^T A\boldsymbol{p}$ was also checked during every iteration, since a non-positive value indicated that the matrix did not satisfy the positive-definiteness requirement. No preconditioner was used.

4.3 Serial optimization

Once the serial baseline had been verified, the most promising opportunities for serial optimization were identified. Mostly, the optimization consisted of removing unnecessary memory allocations, repeated calculations, and repeated traversals of the same data. No targeted serial changes were made to `main.c` or `mesh.c`, since they were deemed to be peripheral to the essential program. The implemented optimizations were therefore confined to:

- `csr.c`: replacement of the dynamically growing row-wise adjacency arrays with a two-pass, contiguously allocated approach;
- `FEM.c`: reuse of element geometry, combined local stiffness and load calculation, fewer searches during assembly, and single-pass boundary-condition application;
- `CG.c`: combined matvec and inner product calculations, together with a convergence test based on squared residual norms.

Also, compiler vectorization diagnostics were inspected before considering more directed SIMD optimization. With `-O3 -march=native`, GCC already vectorized the regular contiguous loops in `CG.c` and some in `csr.c`, with broadly similar coverage in the baseline and optimized implementations. The remaining CSR and FEM kernels contain irregular indirect accesses, reductions, scatter updates, or loop-carried dependencies, making them poor SIMD candidates without substantial changes to the algorithms or data structures. No explicit SIMD directives or intrinsics were therefore introduced.

4.3.1 `csr.c`

The largest and most consistently performing serial improvement was made in `build_csr_pattern()`, where the dynamically growing adjacency arrays and the helper function `append()` were removed. Although dynamic appending was convenient in the baseline because the length of each matrix row did not have to be known in advance, it could require thousands of calls to `realloc()`, while the separately allocated row arrays also may have scattered the temporary adjacency data throughout memory, hurting spatial locality.

The optimized construction instead determines the required temporary storage before inserting any column indices. The array `row_sizes` is first initialized with one entry per row accounting for the diagonal, after which the elements are traversed once and the remaining two off-diagonal entries are counted for each of the triangle's three nodes. A prefix sum of these row sizes forms `temp_row_ptr`, which specifies the beginning of every row in one contiguous temporary array named `adjacency`. The insertion cursor `next[i]` is initialized to the beginning of row i, each diagonal is inserted exactly once, and a second element traversal inserts the six off-diagonal connections contributed by every triangle.

This additional traversal of the contiguous element array should be inexpensive compared with repeated memory reallocation, and it allows the complete temporary adjacency structure to be allocated only once. It also avoids generating repeated diagonal entries for every triangle incident on a node, as was done by the baseline. Repeated off-diagonal entries still occur when several triangles connect the same pair of nodes, and these were handled like in the baseline implementation. The final, correctly sized `row_ptr` and `col_idx` arrays are subsequently constructed, and the compacted rows are copied into `col_idx` using `memcpy()`. Thus, the final CSR pattern remains identical, duplicate-free, and sorted, while its construction uses predictable allocations and a more spatially local memory layout.

4.3.2 `FEM.c`

The first optimization in `FEM.c` concerned the local element calculations. The separate functions `local_stiffness()` and `local_load()` were replaced by `local_stiffness_and_load()`, which computes the coordinate differences and `twice_area` $= 2|K|$ only once

and then reuses them for both local contributions. Since the constant source term gives the same load contribution, $|K|/3$, at all three local nodes, the optimized function returns one scalar `load` rather than constructing a three-entry array containing the same value. This removes the repeated area calculation, reduces temporary storage, and keeps all element-local arithmetic within one function call.

The search for the corresponding positions in the global CSR matrix was also redone. The baseline called `find_position()` separately for all nine entries of every local stiffness matrix, meaning that three independent binary searches were performed in each of the three affected CSR rows. The optimized helper function `find_element_positions()` instead traverses one row sequentially and locates the positions of all three element nodes during that traversal; it is called once for each local row, thereby replacing nine binary searches per triangle with three linear row scans.

Although binary search has better asymptotic search complexity, the CSR rows generated by our mesh are short, and repeatedly restarting a branch-dependent binary search on such small arrays may provide very little advantage, or perhaps even make CPU pipelining more diffuclt. A single sequential traversal accesses `col_idx` contiguously, reuses the row while it is likely to remain in cache, and terminates as soon as all three positions have been found. The returned positions are then used to update the three corresponding entries of `val` directly.

Finally, `apply_dirichlet_bc()` was reduced to one traversal of the CSR rows. For a boundary row, the load entry is set to zero, its diagonal matrix entry is set to one, and every other entry is cleared during the same row traversal; for an interior row, entries whose column corresponds to a boundary node are cleared in order to preserve symmetry. This eliminates the separate diagonal search performed by the baseline and avoids revisiting the boundary rows in a second pass.

The immediate measured benefit of these optimization efforts were less consistent across machines and sessions, which is reasonable because this stage already occupied only a small fraction of the total runtime; it was nevertheless retained.

4.3.3 `CG.c`

The optimized `CG.c` retained the same CG iteration but fused operations that previously required separate traversals of the matrix or working vectors. During initialization, one loop now constructs `r` and `p` while simultaneously accumulating $\boldsymbol{r}^T\boldsymbol{r}$ and $\boldsymbol{f}^T\boldsymbol{f}$. Within every iteration, the new helper function `csr_matvec_dot()` computes $A\boldsymbol{p}$ and $\boldsymbol{p}^TA\boldsymbol{p}$ during the same CSR row traversal, using the row product produced by `csr_row_product()` both as the corresponding entry of `Ap` and as the contribution to the inner product. The updates of $\boldsymbol{x}$ and $\boldsymbol{r}$ are similarly combined with the calculation of the new value of $\boldsymbol{r}^T\boldsymbol{r}$.

These fusions remove two complete vector traversals from every CG iteration and allow newly computed entries of $A\boldsymbol{p}$ and $\boldsymbol{r}$ to be reused immediately, rather than loading them again in subsequent loops. The search-direction update remains separate because its coefficient β depends on the fully accumulated new residual norm and is therefore not known until the preceding traversal has completed.

The convergence test was also rewritten in terms of squared norms. Rather than evaluating a square root after every iteration, the value of $\boldsymbol{r}^T\boldsymbol{r}$ is compared directly with the squared tolerance multiplied by $\boldsymbol{f}^T\boldsymbol{f}$, while the square root is taken only when the

relative residual is reported. Since all quantities involved are non-negative, this produces the same stopping decision while removing an unnecessary operation from the iterative loop.

A manually unrolled version using four independent accumulators was also tested, with the intention of increasing instruction-level parallelism and making better use of vector registers; however, it was consistently slower than the baseline on average and was therefore discarded. In general, the optimization efforts in this module were also generally inconclusive, though they were still kept.

4.4 Parallelization with OpenMP

The serially optimized implementation was used as the starting point for the shared-memory parallelization. OpenMP directives were added only where loop iterations either wrote to non-overlapping memory locations or accumulated scalar quantities that could be handled using reductions. In some places, atomic operations were used. This allowed the existing data structures and overall program organization to be retained.

All parallel loops used `schedule(static)`. The work could therefore be divided between the threads before each loop began, avoiding the additional runtime coordination required by dynamic scheduling. Static scheduling was also suitable because the vector iterations had identical amounts of work, while the CSR rows of the approximately uniform triangular mesh generally had similar lengths. The main changes were:

- `csr.c`: parallelization of the independent row initialization, sorting, duplicate removal, nonzero counting, and final copying operations;
- `FEM.c`: parallelization of the element assembly using atomic global updates, and row-wise parallelization of the boundary-condition application;
- `CG.c`: parallelization of the row products and vector operations, with reductions for inner products and residual norms.

No OpenMP changes were made to `mesh.c`, since its ordered file parsing was not considered a promising target for loop-level parallelization. The parallel executable was compiled with `-fopenmp`, while the number of threads was selected externally using `OMP_NUM_THREADS`. Consequently, the same executable could be used for any thread count without modifying the source code. `main.c` was not altered either, since it was not material to the timing of the algorithm and pipeline.

Note that none of the changes to the code implemented in this version of the program changed the vectorizations present in the prior builds in any meaningful way.

4.4.1 `csr.c`

The two-pass CSR construction contained a mixture of independent row operations and passes with shared updates. OpenMP was therefore applied selectively rather than to the complete function.

The initialization of `row_sizes` was parallelized directly, since each iteration wrote to a separate array entry. The initialization of the insertion cursor `next[i]` and insertion of the diagonal entry of row i were similarly combined in a parallel row loop. Each thread operated on a distinct temporary row, so these writes required no synchronization.

The element traversal used to count the temporary row lengths remained serial. Different elements frequently share nodes and would consequently attempt to increment the same entries of `row_sizes` concurrently. Parallelizing this loop directly would therefore introduce data races. The second element traversal, in which the off-diagonal connections are inserted, was retained serial for the same reason: simultaneous updates of `next[i]` could cause threads to select the same insertion position or skip positions entirely. Neither atomic operations nor thread-private copies were deemed worthwhile implementing here, which will be explained later on.

After all temporary connections had been inserted, the sorting and duplicate removal were parallelized over the matrix rows. Every row occupied a non-overlapping section of `adjacency`, allowing several calls to `qsort()` and the subsequent in-place compactions to proceed concurrently without interference. The number of remaining entries in each row was stored in `row_sizes[i]`, while their sum was accumulated into `total_nnz` using an OpenMP addition reduction.

The prefix sums used to construct `temp_row_ptr` and the final `row_ptr` remained serial because every entry depends on the preceding entry. Although a parallel scan could have been implemented, this would have required a more complicated multi-stage procedure for relatively (performance-wise) uninfluential operations.

Finally, the copying of the compacted temporary rows into `col_idx` was parallelized. Each iteration copied one row using `memcpy()`, and the final row pointers guaranteed that the destination ranges did not overlap. The parallel CSR construction therefore required neither locks nor atomic operations: only the stages that were naturally independent between rows were distributed among the threads.

4.4.2 `FEM.c`

The main element loop in `sparse_global_assemble()` was parallelized using `#pragma omp parallel for` with static scheduling. Each thread independently retrieved an element, located its nodes, and calculated its local stiffness matrix and load contribution. All element-local variables, including `Ke`, `load`, and the element-node indices, were declared inside the loop and were therefore private to the thread executing that iteration.

The subsequent accumulation into the global stiffness matrix and load vector required synchronization because neighbouring elements can contribute to the same global entries. Each update of `f[I]` and `A->val[k]` was therefore protected using `#pragma omp atomic update`. This amounted to three atomic load-vector updates and nine atomic matrix updates per triangle. A version based on thread-private copies of the global matrix and load vector was also considered, but was scrapped for reasons that will be discussed later.

The serial implementation could immediately return when a degenerate triangle was encountered, but returning from inside an OpenMP parallel loop is not permitted. The parallel version instead recorded the smallest index of any degenerate element using a minimum reduction on `first_degenerate_element`. The error was then reported after all threads had completed the loop.

The boundary-condition application was also parallelized over the CSR rows using static scheduling. The existing single-pass row formulation was well suited to this decomposition: iteration i modified only `f[i]` and the nonzero entries belonging to row i, which are disjoint from the entries belonging to every other row. Consequently, no atomic operations

were required. The detection of a missing diagonal entry was handled using a minimum reduction on `first_missing_diagonal`, after which any error was reported outside the parallel loop.

4.4.3 `CG.c`

The principal, and indeed most impactful, OpenMP parallelization was performed in `CG.c`. The outer loop of `csr_matvec()` was distributed between the threads because the calculation of each output entry depends only on its corresponding CSR row. The matrix and input vector were shared and read-only during the operation, while the row sum was private to the thread executing that iteration and the resulting value was written only to the corresponding entry of the output vector.

The fused function `csr_matvec_dot()` was parallelized in the same manner. Every thread computed a subset of the entries of $A\boldsymbol{p}$ and accumulated its corresponding partial contribution to $\boldsymbol{p}^T A\boldsymbol{p}$. The partial inner products were then combined using an OpenMP addition reduction.

The initialization loop for the working vectors was parallelized with simultaneous reductions for $\boldsymbol{r}^T\boldsymbol{r}$ and $\boldsymbol{f}^T\boldsymbol{f}$. Within each iteration, the combined update of $\boldsymbol{x}$ and $\boldsymbol{r}$ was distributed over the vector indices, while the new value of $\boldsymbol{r}^T\boldsymbol{r}$ was obtained using another reduction. The search-direction update required no reduction because every iteration wrote to one distinct entry of `p`.

The reductions may combine their partial sums in a different order for different thread counts. Since floating-point addition is not associative, this can produce small round-off differences even though the mathematical operations and convergence criterion remain unchanged.

5 Performance experiments

This section will outline the experimental methodology used to obtain the results.

5.1 Experimental platform

All performance experiments were conducted on `vitsippa.it.uu.se`, a shared student server operated by Uppsala University. The machine contains two 16-core AMD Opteron 6282SE processors based on the Bulldozer architecture, each operating at a nominal frequency of 2.6 GHz, giving 32 processor cores in total. It has 128 GB of main memory and runs 64-bit Ubuntu Linux (Uppsala University 2026).

The cache hierarchy, as reported by `lscpu`, comprises 512 KiB of L1 data cache across 32 instances, 1 MiB of L1 instruction cache across 16 instances, 32 MiB of L2 cache across 16 instances, and 24 MiB of L3 cache across four instances. These values are aggregate capacities distributed across the machine rather than unified caches accessible equally by every core.

The executables were compiled with GCC 13.3.0 using `-O3`, `-march=native`, `-Wall`, `-Wextra`, and `-std=c11`; the parallel executable was additionally compiled with `-fopenmp`. The thread count was selected using `OMP_NUM_THREADS`. No explicit thread-binding policy was applied, leaving thread placement to the OpenMP runtime and operating-system

scheduler. The 32-thread configuration therefore requested one thread per documented processor core, although the threads were not explicitly pinned to particular cores or sockets. Since the server was shared with other users, concurrent system activity was a potential source of run-to-run variability.

The operating system exposed the machine as four non-uniform memory access (NUMA) nodes, each containing eight processor cores and approximately 32 GB of local memory. In a NUMA system, all cores can access the complete shared memory, but accessing memory associated with another NUMA node is generally more costly than accessing local memory. Since neither the OpenMP threads nor the memory allocations were explicitly bound to particular NUMA nodes, a 32-thread execution would normally span all four nodes, while smaller thread teams could be confined to one, or distributed across a subset of nodes.

5.2 Serial Experiments

The baseline and serially optimized executables were run directly from the terminal on `vitsippa`. Both executables were tested using the same mesh of approximately 400 000 nodes, selected to keep the experimental runtime manageable. Each executable was run five times, and the median timing for each measured module was used as its representative runtime because it was less sensitive to occasional system noise. Mesh-reading time was excluded from the module-by-module comparison, since it was never optimized, but remained included in the pipeline runtime. This choice was made in order to show the realistic, practical runtime of the entire program; the influence of `mesh.c` is minimal, and the results are still reported for each optimized stage.

5.3 Parallel Experiments

The parallel experiments were performed on `vitsippa`. A Bash script was used to automate the runs and record the CG runtime, total runtime, iteration count, and whether the solver completed successfully. Each configuration was run five times. The meshes were generated using `disk.geo`, with the uniform mesh size varied to obtain the required problem sizes.

For the strong-scaling experiment, the problem size was fixed at 1 615 235 mesh nodes, while the number of OpenMP threads was increased through 1, 2, 4, 8, 16, and 32.

For the weak-scaling experiment, the problem size was increased approximately in proportion to the thread count, maintaining roughly 100 000 mesh nodes per thread. The tested configurations were 101 507 nodes with one thread, 200 938 nodes with two threads, 401 848 nodes with four threads, 805 439 nodes with eight threads, and 1 615 235 nodes with 16 threads.

For each configuration across both experiments, the median values out of the five obtained runtimes were used for visualization and analysis. This is because using the average is susceptible to being skewed by outliers with massively slower runtimes, which can be caused by arbitrary background processes on the computer. This approach yielded the most representative data for the implementation on `vitsippa`.

6 Results

In this section, the results of the performance experiments will be presented. All plots and visualizations were done in Python using `matplotlib`.

6.1 FEM solver numerical results

For the problem $-\Delta u = 1$ on the unit disk with homogeneous Dirichlet boundary conditions, the analytical solution is

$$u(x, y) = \frac{1 - x^2 - y^2}{4}.$$

Figure 1 compares this solution with the finite-element solution obtained using 1 615 235 nodes and 16 OpenMP threads.

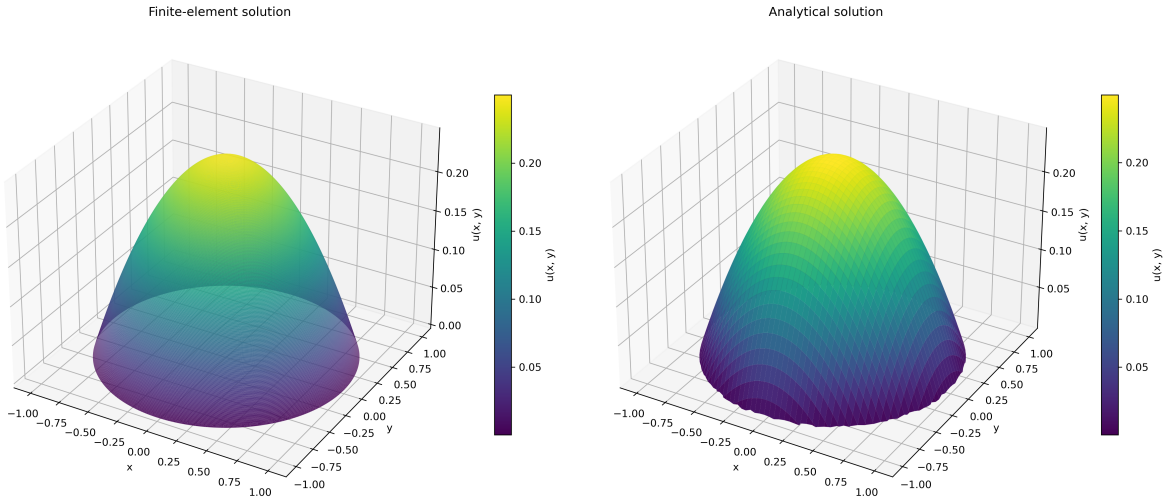


Figure 1: Finite-element solution (left) and analytical solution (right).

The corresponding correctness data are presented in Table 1. The residual was independently recomputed as $\|f - Ax\|_2/\|f\|_2$, while the nodal error was evaluated against the analytical solution.

Table 1: Representative correctness results for 1 615 235 nodes and 16 threads.

Quantity	Value
CG iterations	2117
Reported relative residual	$9.919297236977 \times 10^{-11}$
Recomputed relative residual	$5.193858372912 \times 10^{-10}$
Largest boundary solution magnitude	0
Largest nodal error	$1.037090522845 \times 10^{-7}$
Correctness test	Passed

This result was representative of the experiments performed throughout development and testing, all of which passed the correctness test.

6.2 Serial Results

All five runs of both executables completed successfully and passed the CG correctness test. The baseline executable required 1280 CG iterations, while the serially optimized executable required 1279 iterations.

The speedup for each timed module was calculated from the corresponding median runtimes as

$$S = \frac{T_{\text{baseline}}}{T_{\text{optimized}}}.$$

Table 2 presents the median runtimes and resulting speedups. Mesh reading is omitted as an individual module but remains included in the pipeline total.

Table 2: Median runtimes of the baseline and serially optimized executables and the resulting speedups for a mesh of approximately 400 000 nodes.

Timed module	Median runtime [s]		Speedup
	Baseline	Optimized	
CSR-pattern construction	0.865911569	0.321393498	2.694
Vector allocation	0.000506908	0.000125884	4.027
FEM assembly	0.496916216	0.515133810	0.965
Boundary conditions	0.010140823	0.008318297	1.219
CG solver	24.595480794	23.415956764	1.050
Compute total	25.963545023	24.254933589	1.070
Pipeline total	26.710074483	25.010827713	1.068

6.3 Parallel Results

The reported values were calculated from the median runtime of the five repetitions for each configuration. Let T_p^{CG} and T_p^{tot} denote the CG-only (meaning, only the time taken by the CG algorithm) and full-pipeline runtimes with p threads, respectively, and let k_p denote the corresponding number of CG iterations.

6.3.1 Strong Scaling

For strong scaling, the ordinary speedup and the speedup based on runtime per CG iteration were calculated for each runtime measure T as

$$S_p(T) = \frac{T_1}{T_p}, \qquad \widehat{S}_p(T) = \frac{T_1/k_1}{T_p/k_p}.$$

The corresponding parallel efficiencies were calculated as

$$E_p(T) = \frac{S_p(T)}{p}, \qquad \widehat{E}_p(T) = \frac{\widehat{S}_p(T)}{p}.$$

The iteration-normalized quantities were calculated for both the CG-only runtime and the total pipeline runtime. Table 3 gives the strong-scaling results for the fixed problem size of 1 615 235 nodes.

Table 3: Strong-scaling speedups, efficiencies, and CG iteration counts calculated from the median runtimes. The hatted CG quantities are based on runtime per CG iteration.

p	k_p	S_p		$\widehat{S}_p$	E_p		$\widehat{E}_p$
		CG	tot	CG	CG	tot	CG
1	2559	1.000	1.000	1.000	1.000	1.000	1.000
2	2555	1.599	1.588	1.597	0.800	0.794	0.798
4	2134	3.289	3.197	2.743	0.822	0.799	0.686
8	2115	6.050	5.704	5.000	0.756	0.713	0.625
16	2115	8.671	7.973	7.166	0.542	0.498	0.448
32	2113	8.003	7.444	6.608	0.250	0.233	0.207

Figure 2 shows the ordinary and iteration-normalized speedups for the CG-only and full-pipeline runtimes.

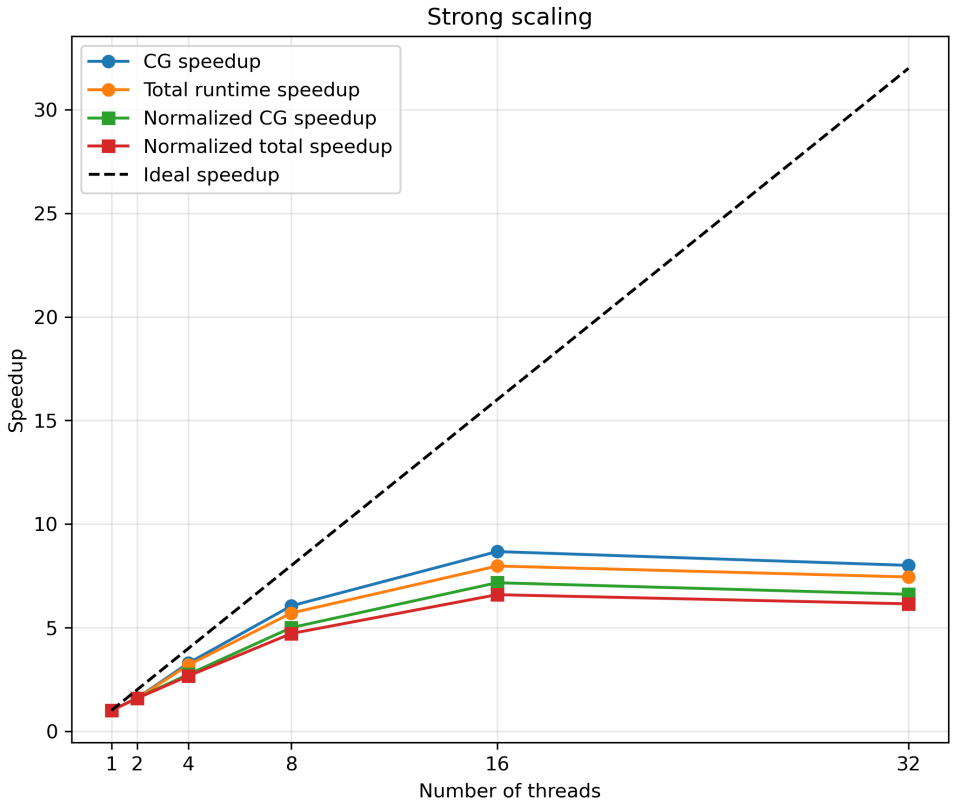


Figure 2: Strong-scaling speedups based on the median CG-only and full-pipeline runtimes. The normalized curves are calculated from runtime per CG iteration, and the dashed line represents ideal speedup.

6.3.2 Weak Scaling

For weak scaling, the efficiencies based on the ordinary runtime and the runtime per CG iteration were calculated as

$$E_p(T) = \frac{T_1}{T_p}, \qquad \widehat{E}_p(T) = \frac{T_1/k_1}{T_p/k_p},$$

where the one-thread weak-scaling configuration was used as the reference. The iteration-normalized efficiencies were calculated for both the CG-only runtime and the total runtime.

Table 4 gives the weak-scaling results for the problem sizes increased approximately in proportion to the thread count, so as to keep the supposed load per thread constant.

Table 4: Median weak-scaling runtimes, CG iteration counts, and resulting efficiencies. The hatted CG efficiency is based on CG runtime per iteration.

p	Nodes	Median runtime [s]		k_p	E_p		$\widehat{E}_p$
		CG	tot		CG	tot	CG
1	101 507	2.099	2.521	713	1.000	1.000	1.000
2	200 938	5.406	6.109	973	0.388	0.413	0.530
4	401 848	8.612	9.817	1266	0.244	0.257	0.433
8	805 439	16.057	18.262	1666	0.131	0.138	0.305
16	1 615 235	28.648	32.751	2117	0.073	0.077	0.218

Figure 3 shows the ordinary and iteration-normalized efficiencies for the CG-only and full-pipeline runtimes.

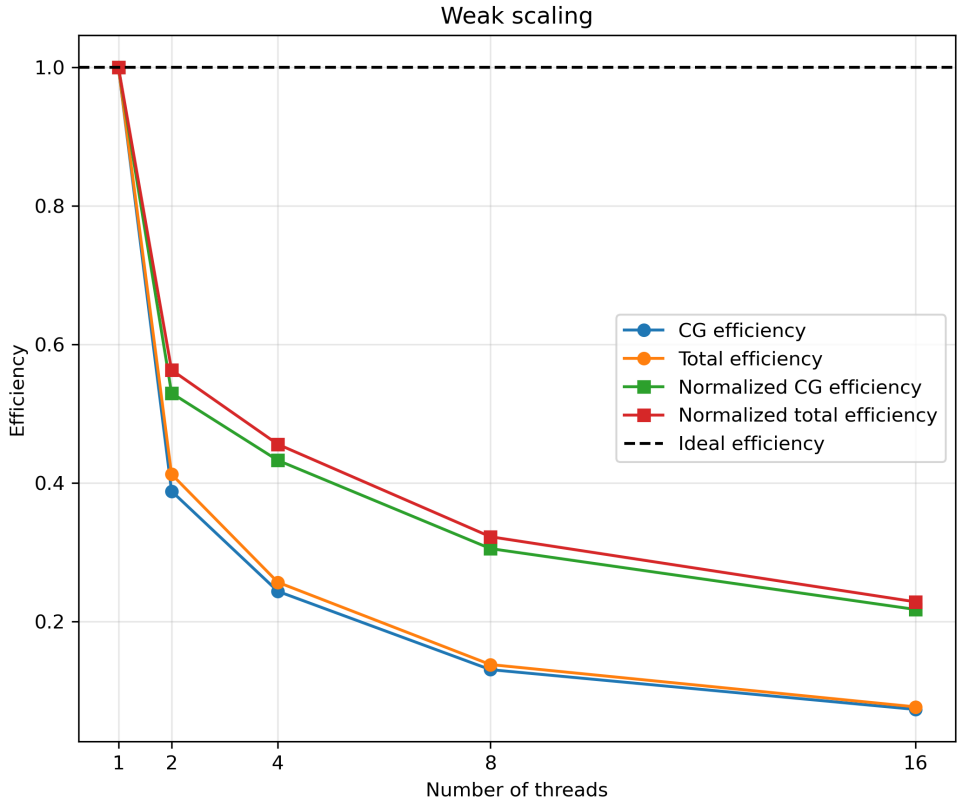


Figure 3: Weak-scaling efficiencies based on the median CG-only and full-pipeline runtimes. The normalized curves are calculated from runtime per CG iteration, and the dashed line represents ideal efficiency.

7 Discussion

In this section, we will discuss the different stages of the implementation and their performance results. Specifically, we will explore the motivations behind why certain optimization and parallelization choices were made. We will also touch upon the limitations of the optimizations and in particular the shared-memory parallelization, at least on this type of problem.

Note that any numbers discussed in the following subsections that are not present in any of the tables included in the previous sections, are presented along with the full raw experimental data in Appendix A.

7.1 Serial optimization

The serial optimization was performed incrementally. Relatively simple changes were considered first, followed by increasingly structural modifications to the CSR construction, finite-element assembly, and CG solver. Correctness and runtime were evaluated after each change. Both the baseline and optimized versions were compiled using `-O3 -march=native`, meaning that transformations also had to improve upon the compiler's own optimizations. The purpose of this stage was therefore both to reduce serial runtime and to establish a suitable baseline for the subsequent OpenMP parallelization.

7.1.1 `csr.c`

The clearest improvement was obtained in the CSR-pattern construction. Its runtime decreased from 0.8659 s to 0.3214 s, corresponding to a speedup of 2.694 and a runtime reduction of approximately 62.9%. This result was also reproducible during the earlier exploratory measurements on different machines and in different sessions. The consistency is reasonable because the optimization removed a substantial and identifiable source of overhead. The baseline generated nine temporary adjacency entries per triangle and expanded a separately allocated array for each matrix row using repeated calls to `realloc()`. In contrast, the optimized implementation first determined the row sizes and then stored the temporary structure in one contiguous allocation.

The traversal of the element array required by the two-pass construction was therefore probably faster than the allocation and copying during dynamic array growth. Contiguous storage might've also improved spatial locality and removed the fragmentation associated with many independently allocated row arrays. Since the final sorting, duplicate removal, and resulting CSR pattern remained equivalent to those of the baseline, the improvement can reasonably be attributed to the more efficient construction. This indicates that performing an inexpensive preliminary pass can be beneficial when it makes the later memory requirements predictable.

Despite its large relative speedup, CSR construction represented only about 3.34% of the baseline compute time. The absolute saving was approximately 0.545 s, and even eliminating the stage entirely could only have produced a compute speedup of about 1.035. The CSR optimization was consequently successful as an isolated transformation, but could not by itself produce a large end-to-end improvement. This illustrates the importance of considering both the relative speedup of an individual stage and its contribution to the total runtime of a program, as it can save time and resources in a production situation.

7.1.2 `CG.c`

The CG solver presented the opposite situation to the CSR construction. Its speedup was only 1.050, but it accounted for approximately 94.7% of the baseline compute time. Reducing its median runtime from 24.5955 s to 23.4160 s therefore saved approximately 1.180 s, more than twice the absolute saving obtained from CSR construction. The baseline and optimized versions required 1280 and 1279 iterations, respectively. Normalizing by these counts gives average iteration times of approximately 19.215 ms and 18.308 ms, corresponding to a per-iteration speedup of 1.0496. Thus, the measured CG improvement cannot in this case be explained by a difference in iteration counts.

The motivation for loop fusion in CG was to reduce memory traffic. Combining the sparse matrix-vector product with the dot product of $p^T(Ap)$ eliminated a subsequent traversal of p and Ap, while combining the updates of x and r with the calculation of $r^T r$ eliminated another traversal of the residual vector. The updated values could instead be reused while ideally still held in registers or cache. The squared convergence test additionally removed a square-root evaluation from each iteration, although this arithmetic saving was likely negligible compared with the memory operations.

The limited and previously inconsistent benefit of the CG changes indicates a trade-off. The removed vector traversals access memory contiguously and may be vectorized efficiently, whereas the CSR matrix-vector product contains indirect accesses through

`col_idx`. Fusing the dot product into this irregular kernel reduces memory traffic, but may also make SIMD vectorization and instruction scheduling more difficult than in a separate, simple dot-product loop. Earlier exploratory measurements on Vitsippa varied from an improvement of approximately 0.5 s to a slowdown of approximately 0.2 s, while measurements on Arrhenius showed no clear difference. The balance between saved memory traffic and lost vector efficiency may depend on the processor, cache state, memory-bandwidth pressure, and other activity on the system. Without hardware-counter measurements, these explanations remain hypotheses; nevertheless, the final five-run median indicates a modest benefit on Vitsippa. However, one small proposed benefit of this was providing a cleaner base for OpenMP parallelization, as fusing two loops into one removes the need for an extra synchronization point.

A four-way manually unrolled CG implementation was also tested to increase instruction-level parallelism and shorten the dependency chains in the reductions. It instead made the solver approximately one second slower on average and was discarded. One plausible explanation is that the compiler already performed suitable unrolling and vectorization for the regular loops, while the manual transformation increased register pressure and loop-body size without resolving the indirect memory access of the sparse matrix-vector product.

7.1.3 `FEM.c`

The FEM assembly optimizations did not produce a clear improvement in the final experiment. Its median runtime *increased* from 0.4969 s to 0.5151 s, corresponding to a speedup of 0.965, or a slowdown of approximately 3.7%. Earlier exploratory runs had occasionally shown a small improvement, but this was not consistent between machines or sessions. The final results therefore do not support claiming a reliable assembly speedup.

The uncertain outcome is understandable because the removed work was relatively small. Reusing the triangle area eliminates only a few arithmetic operations per element, while both versions must still calculate and accumulate the nine stiffness contributions. Similarly, replacing nine binary searches with three sequential row traversals could improve locality in principle, but the CSR rows of a two-dimensional triangular mesh are short. Binary search over such rows is already inexpensive, while a linear traversal may perform several comparisons before locating all three entries. Consequently, the relative performance can depend on row lengths, compiler decisions, and cache state. The optimized formulation was retained because it avoids objectively redundant calculations, simplifies the element-local data flow, and had no negative effect on the complete runtime on average.

The single-pass boundary-condition implementation reduced its runtime from 0.01014 s to 0.00832 s, a relative reduction of approximately 18.0%. This agrees with the intended removal of the second traversal and the separate diagonal search. However, the absolute saving was only 1.82 ms, making the result particularly sensitive to timing noise and irrelevant to the total runtime. The reported 4.027 speedup for vector allocation should be interpreted similarly. Interestingly, no targeted optimization was made to this operation, and the absolute difference was only 0.381 ms. Altered allocator state as a result of the prior memory-oriented optimizations, or ordinary measurement variation could be plausible explanations.

7.2 Parallelization

The OpenMP implementation produced a substantial reduction in time to solution, but its scalability was ultimately limited, most likely by the inherent properties of both the CG solver and the shared-memory system. Parallelization was applied selectively to operations involving independent rows, elements, and vector entries that were part of computation kernels that were considered to be dominant across the program build. The uniform triangular meshes provide reasonably balanced work for `schedule(static)`, and because of this, dynamic scheduling was thought to have introduced unnecessary coordination overhead to an otherwise well load-balanced setup. The resulting implementation scaled reasonably effectively (in the strong sense) up to approximately 8–16 threads, after which additional threads no longer produced a dependable improvement; indeed, moving from 16 to 32 threads made the runtime slightly longer on average.

In this section, the modules will be discussed individually: only `CG.c` will touch on the results as presented in section 6, while all other modules will instead deal with parallelization strategy choices. This is because, in essence, the runtime of the entire program build is attributed to the CG algorithm; practically, to optimize and parallelize this program for runtime performance is tantamount to doing it for the CG algorithm.

7.2.1 `csr.c`

The two element traversals in the CSR construction were kept serial because directly distributing either traversal would introduce data races. In the first traversal, neighboring elements can update the same entry of `row_sizes`; lost increments could underestimate the required row lengths and consequently make the allocated temporary row ranges too small. In the second traversal, concurrent updates to the insertion cursors in `next` could assign the same adjacency position to multiple entries, causing valid column indices to be overwritten and leaving other positions uninitialized. The subsequent sorting and duplicate-removal stage cannot recover entries lost in this manner. The prefix-sum loops that construct `temporary_row_ptr` and `A->row_ptr` were also kept serial because every iteration depends on the offset calculated by the preceding iteration and therefore cannot be distributed using a simple `omp parallel for`. These restrictions to parallelization are not final: atomic operations could protect the row-size updates and reserve unique adjacency positions, or thread-private counting schemes could avoid the atomics, and a multi-stage parallel scan could replace the serial prefix sums. However, atomics would introduce synchronization and contention, while the private schemes would require $\mathcal{O}(pN)$ additional storage together with a reduction; a parallel scan would likewise add multiple barriers. Particularly, the memory load of the thread-private copies could become especially memory-intense for larger meshes. Since CSR construction contributed only a small fraction of the complete runtime, the expected end-to-end benefit did not justify this additional memory usage, synchronization, and implementation complexity. Parallelization was therefore limited to the row-independent stages, including initialization, sorting, duplicate removal, and final copying, for which distinct rows occupy non-overlapping memory ranges in the data structures they are stored in.

7.2.2 `FEM.c`

Finite-element assembly presented a different synchronization problem. The element-local calculations are independent, but neighboring elements contribute to the same entries of

the global stiffness matrix and load vector. The chosen implementation protected the three load-vector updates and nine matrix updates per triangle using atomic operations. This retained the existing element-wise decomposition and required little additional memory, but concurrent updates to shared nodes can serialize and generate cache-coherence traffic, as well as cause the contention and synchronization overhead normally associated with atomic operations. Thread-private copies of the complete matrix and load vector would have avoided these atomics, but their memory requirement would scale as $\mathcal{O}(p\,\mathrm{nnz})$, followed by an additional reduction of the private copies. This was considered an unsuitable trade-off for larger meshes. Since assembly represented only a small fraction of the total runtime, even a substantially more sophisticated strategy, such as element coloring, would probably have had limited influence on the complete pipeline. The reason as to why atomics were used here but not in `csr.c` was simply to test if the atomics hurt the average runtime of the FEM assembly, which, after comparison to a build whose only difference was the absence of the same atomics, showed no difference in runtime at all. The atomics were kept, really, as an arbitrary decision.

7.2.3 `CG.c` and strong- and weak-scaling behavior

The principal parallel benefit came from the CG solver, which dominated the complete runtime. Its median speedups at 2, 4, 8, and 16 threads were 1.599, 3.289, 6.050, and 8.671, respectively. The corresponding full-pipeline speedups were slightly lower at 1.588, 3.197, 5.704, and 7.973 because the remaining program stages contributed additional serial and weakly parallelized work. The best median time was obtained with 16 threads. Increasing the thread count to 32 reduced the CG speedup to 8.003 and the full-pipeline speedup to 7.444. The ordinary CG efficiency consequently decreased from 75.6% at eight threads to 54.2% at 16 threads and 25.0% at 32 threads. The implementation therefore provided useful strong scaling through approximately 16 threads, after which performance saturated.

The raw speedups must nevertheless be interpreted together with the CG iteration counts. For 1 through 32 threads, these were

$$2559, \quad 2555, \quad 2134, \quad 2115, \quad 2115, \quad 2113.$$

OpenMP reductions combine their partial sums in a thread-dependent order, while the atomic FEM assembly can similarly change the order in which element contributions are accumulated. Since floating-point addition is not associative, these changes can alter the CG trajectory and iteration count without compromising the correctness of the final solution. The parallel runs therefore benefited partly from performing fewer iterations.

After normalizing by the corresponding iteration counts, the CG speedups at 8, 16, and 32 threads become 5.000, 7.166, and 6.608, respectively, with efficiencies of 62.5%, 44.8%, and 20.7%. The normalized efficiencies decrease consistently as the thread count increases and show that the loss of parallel efficiency is real rather than solely an effect of differing convergence histories. The raw quantities remain the appropriate measures of time to solution, whereas the normalized quantities provide a diagnostic measure of the cost of executing one CG iteration.

A plausible primary explanation for the high-thread saturation is pressure on the memory system. For the fixed $N = 1\,615\,235$ problem, one length-N `double` vector occupies

$$8N \approx 12.3\,\mathrm{MiB}.$$

The four vectors repeatedly accessed during the CG iterations –`x`, `r`, `p`, and `Ap`– therefore occupy approximately 49.3 MiB, while all six allocated vectors occupy approximately 73.9 MiB. Assuming approximately seven stored matrix entries per interior row, the CSR values, column indices, and row pointers add roughly 136 MiB. The active CG working set is consequently on the order of 185 MiB.

This working set substantially exceeds Vitsippa's 24 MiB of aggregate L3 cache, which is itself distributed across four separate cache instances. The complete matrix and vector data therefore cannot remain cache-resident, and substantial main-memory traffic during every CG iteration is unavoidable. This does not mean that every individual access misses in cache, but the complete working set must be repeatedly streamed through the memory hierarchy.

The dominant CSR matrix–vector product also has low arithmetic intensity. Each nonzero requires a matrix value and column index to be read, followed by an indirect access to the input vector, for only a multiplication and an addition. Increasing the number of threads therefore increases the simultaneous demand for memory traffic much more rapidly than it increases useful floating-point work. Once the available memory bandwidth is approached, additional threads compete for the same shared resource and cannot produce proportional speedup. The cache-exceeding working set, low arithmetic intensity, declining iteration-normalized efficiency, and lack of improvement beyond 16 threads together make memory-bandwidth contention a strong explanation supported by the available evidence.

Though, achieved bandwidth and cache-miss rates were not measured, so bandwidth saturation cannot be demonstrated definitively or separated quantitatively from the other overheads.

Vitsippa additionally exposes 4 NUMA nodes each containing 8 CPU cores. Several large arrays are allocated or initialized sequentially, which may place many pages near one NUMA node under a first-touch policy. Threads running elsewhere could then generate remote-memory traffic. Because 32 threads would have to occupy all NUMA nodes, this might be a reason for the unreliable performance at 32 threads – though, smaller thread teams might still be distributed across all 4 nodes, so this remains speculative. In any case, exact thread placement, page placement, and NUMA-locality data were not recorded during experiments, so this remains a plausible contributor to scaling limitation rather than a conclusive one.

Synchronization behavior is another source of scaling limitation. Each CG iteration contains two OpenMP reductions and several separate work-sharing loops with implicit barriers. These loops are also implemented as separate parallel regions, producing repeated fork–join overhead. As strong scaling reduces the work assigned to each thread, these fixed synchronization costs become a larger fraction of the iteration time.

The high-thread measurements were also variable. The 16-thread CG runtimes ranged from approximately 28.015 to 39.242 s, while the 32-thread runtimes ranged from 29.424 to 45.853 s. The $N = 1\,615\,235$, $p = 16$ configuration appeared in both campaigns and produced medians of 34.660 and 28.648 s, although the individual ranges overlapped. The overall conclusion of the strong scaling campaign is therefore that performance saturates around 16 threads and that 32 threads provide no reliable improvement, and that a likely explanation is memory pressure.

Weak scaling exposes two separate limitations: increasing CG iteration counts and an

increasing cost per iteration. Although the number of mesh nodes per thread was kept approximately constant, the CG iteration counts increased as

$$713, \quad 973, \quad 1266, \quad 1666, \quad 2117,$$

corresponding to a factor of

$$\frac{2117}{713} \approx 2.97$$

between one and 16 threads. This increase is primarily associated with mesh refinement and the worsening conditioning of the unpreconditioned stiffness matrix. Floating-point accumulation order also contributes to the exact counts, particularly for the largest meshes.

Iteration normalization isolates this convergence effect from the cost of executing the iterations. The median CG time per iteration increased from approximately 2.944 ms for the one-thread problem to 13.532 ms for the 16-thread problem, a factor of approximately 4.60. The complete runtime increase can therefore be decomposed as

$$\frac{2117}{713}\frac{13.532}{2.944} \approx 2.97 \times 4.60 \approx 13.65.$$

Consequently, the iteration-normalized CG efficiency at 16 threads was 21.8%, compared with the raw efficiency of 7.3%. Nearly threefold iteration count explains part of the poor weak scaling, but the remaining 4.60-fold increase in time per iteration demonstrates a separate implementation and memory-system limitation.

The largest relative increase in time per iteration occurred from one to two threads, where it grew by approximately 1.89. This is exceptionally poor weak scaling. A plausible explanation for this jump is that the smallest problem may have had substantially better cache residency and locality than all of the larger problems. Maintaining constant mesh nodes per thread does not preserve cache behavior because the threads operate on one increasingly large, globally coupled sparse system rather than on independent local problems. The continuing deterioration beyond two threads indicates that the result cannot be explained by a single cache-capacity transition alone; increasing main-memory traffic, bandwidth contention, NUMA effects, and synchronization continue to limit performance as the global system grows, albeit not as aggressively as the proposed cache limitation in going from 1 to 2 threads in the weak campaign.

7.2.4 Identified parallelization bottlenecks and proposed improvements

The results highlight a combination of two distinct probable (but not definitively determined) limitations:

1. The non-preconditioned algorithm currently in use requires more iterations as the mesh is refined, leading to longer runtimes.
2. The shared-memory implementation becomes less efficient at executing each iteration as the global working set and thread count increase, mainly because of heavier memory loads. Arrays extend into main memory, potentially giving rise to memory-bandwith contention by increasing thread counts; and problem sizes crossing the cache boundary might experience aggressive slowdown as they start using main memory.

An effective preconditioner would address the first limitation and is likely the most important, or if anything, the most accessible algorithmic improvement for time to solution.

The second supposed limitation is more difficult to mitigate, since it appears platform-specific. Though, a more thorough adaptation of the program to platform's specifications, especially in regard to memory hierarchies and layouts, might be a way to bring that limitation down to its minimum.

8 Conclusions

The program build did what it was supposed to do, successfully. It calculated a solution to the Poisson equation on the unit disk with homogeneous Dirichlet boundary conditions, with high accuracy as measured against the analytical solution.

Taken together, the serial changes reduced the compute time from 25.9635 s to 24.2549 s on a roughly $400,000$-node mesh, corresponding to a speedup of 1.070 and a runtime reduction of 6.58%. The full pipeline decreased from 26.7101 s to 25.0108 s, giving a speedup of 1.068. The overall improvement was therefore modest, and whether it was meaningful in relation to the effort of its implementation is not entirely clear. Almost all of the benefit originated from the reproducible CSR restructuring and the smaller relative improvement of the runtime-dominating CG solver, while the remaining stages had little influence on time to solution.

Overall, the OpenMP parallelization was what mattered, and it was successful in reducing the runtime of the large fixed problem, with the best measured full-pipeline speedup being approximately 7.97 at 16 threads. Whatever the scaling says, that is the difference between nearly one full work-day and a lunch, or one workday and an entire week.

The strong-scaling results demonstrate useful shared-memory parallelism, but also show that additional threads beyond 16 might be subject to memory-bandwidth limits, as well as the traditional overhead associated with synchronization. The weak-scaling results show that increasing the problem size introduces both additional CG iterations and a higher cost per iteration, likely due to both deteriorating memory optimality and parallelization overhead. Larger improvements would therefore require algorithmic modifications – of which a preconditioner for the CG solver presents a promising first choice – and adaptations to the specific platform's memory-system.

References

Bell, Nathan and Michael Garland (2009). "Implementing Sparse Matrix–Vector Multiplication on Throughput-Oriented Processors". In: *Proceedings of the Conference on High Performance Computing Networking, Storage and Analysis*. ACM. DOI: 10.1145/1654059.1654078.

Chen, Long (n.d.). *Sobolev Spaces and Elliptic Equations*. Lecture notes.

Gutknecht, Martin H. (2007). "A Brief Introduction to Krylov Space Methods for Solving Linear Systems". In: *Frontiers of Computational Science*. Ed. by Yukio Kaneda, Hiroshi Kawamura, and Masaki Sasai. Berlin, Heidelberg: Springer. DOI: 10.1007/978-3-540-46375-7_5. URL: https://people.math.ethz.ch/~mhg/pub/biksm.pdf.

Liu, Wing Kam, Shaofan Li, and Harold S. Park (2022). "Eighty Years of the Finite Element Method: Birth, Evolution, and Future". In: *Archives of Computational Methods in Engineering* 29, pp. 4431–4453. DOI: 10.1007/s11831-022-09740-9.

OpenMP Architecture Review Board (2024). *OpenMP Application Programming Interface, Version 6.0*. OpenMP Architecture Review Board. URL: https://www.openmp.org/wp-content/uploads/OpenMP-API-Specification-6-0.pdf (visited on 08/06/2026).

Uppsala University (2026). *Linux Servers*. https://www.it.uu.se/datordrift/maskinpark/linux.html. Department of Information Technology; accessed 8 August 2026.

Viaclovsky, Jeff (2004). *Differential Analysis: Weak Derivatives and Sobolev Spaces*. MIT OpenCourseWare lecture notes.

Wikipedia contributors (2026a). *Conjugate gradient method*. Wikipedia, The Free Encyclopedia. URL: https://en.wikipedia.org/wiki/Conjugate_gradient_method (visited on 08/06/2026).

– (2026b). *Finite element method*. Wikipedia, The Free Encyclopedia. URL: https://en.wikipedia.org/wiki/Finite_element_method (visited on 08/05/2026).

– (2026c). *Galerkin method*. Wikipedia, The Free Encyclopedia. URL: https://en.wikipedia.org/wiki/Galerkin_method (visited on 08/06/2026).

– (n.d.). *Weak derivative*. Wikipedia, The Free Encyclopedia. Accessed August 5, 2026.

A Raw experiment data

The raw performance experiment results are presented in Tables 5, 6, and 7.

Table 5: Raw Experimental Data

Type	Threads	Nodes	Rep	CG Time (s)	Total Time (s)	Success	Iter
strong	1	1615235	1	300.519	309.231	1	2559
strong	1	1615235	2	300.642	309.361	1	2559
strong	1	1615235	3	296.419	305.092	1	2559
strong	1	1615235	4	301.074	309.696	1	2559
strong	1	1615235	5	299.806	308.418	1	2559
strong	2	1615235	1	187.896	194.705	1	2555
strong	2	1615235	2	170.000	177.889	1	2555
strong	2	1615235	3	171.038	178.067	1	2555
strong	2	1615235	4	194.324	201.316	1	2555
strong	2	1615235	5	189.632	196.493	1	2555
strong	4	1615235	1	89.820	95.111	1	2134
strong	4	1615235	2	93.676	98.949	1	2134
strong	4	1615235	3	90.963	96.302	1	2134
strong	4	1615235	4	94.349	99.681	1	2134
strong	4	1615235	5	91.379	96.715	1	2134
strong	8	1615235	1	49.671	54.214	1	2115
strong	8	1615235	2	49.725	54.527	1	2115
strong	8	1615235	3	49.399	54.100	1	2115
strong	8	1615235	4	49.563	54.166	1	2115
strong	8	1615235	5	49.754	54.301	1	2115
strong	16	1615235	1	38.462	42.628	1	2115
strong	16	1615235	2	28.015	32.094	1	2115
strong	16	1615235	3	39.242	43.374	1	2115
strong	16	1615235	4	28.196	32.310	1	2115
strong	16	1615235	5	34.660	38.786	1	2115
strong	32	1615235	1	29.424	33.418	1	2113
strong	32	1615235	2	37.550	41.543	1	2113
strong	32	1615235	3	37.931	41.942	1	2113
strong	32	1615235	4	31.338	35.298	1	2113
strong	32	1615235	5	45.853	49.822	1	2113
weak	1	101507	1	2.099	2.523	1	713
weak	1	101507	2	2.096	2.519	1	713
weak	1	101507	3	2.090	2.513	1	713
weak	1	101507	4	2.107	2.530	1	713
weak	1	101507	5	2.100	2.521	1	713
weak	2	200938	1	5.355	6.060	1	973
weak	2	200938	2	5.406	6.111	1	973
weak	2	200938	3	5.406	6.109	1	973
weak	2	200938	4	5.444	6.145	1	973
weak	2	200938	5	5.321	6.019	1	973

Table 5: Raw Experimental Data

Type	Threads	Nodes	Rep	CG Time (s)	Total Time (s)	Success	Iter
weak	4	401848	1	8.625	9.855	1	1266
weak	4	401848	2	8.612	9.817	1	1266
weak	4	401848	3	8.561	9.772	1	1266
weak	4	401848	4	8.575	9.792	1	1266
weak	4	401848	5	8.629	9.822	1	1266
weak	8	805439	1	16.000	18.204	1	1666
weak	8	805439	2	16.121	18.308	1	1666
weak	8	805439	3	16.057	18.262	1	1666
weak	8	805439	4	16.075	18.262	1	1666
weak	8	805439	5	15.871	18.083	1	1666
weak	16	1615235	1	28.579	32.652	1	2117
weak	16	1615235	2	28.648	32.751	1	2117
weak	16	1615235	3	28.044	32.129	1	2117
weak	16	1615235	4	32.887	36.972	1	2117
weak	16	1615235	5	38.762	42.899	1	2117
serial_ref	1	200938	1	7.840	8.753	1	975
serial_ref	1	401848	1	25.097	27.185	1	1279
serial_ref	1	805439	1	95.401	99.511	1	1930

Table 6: Raw performance and correctness results for the baseline build.

Performance timings

Run	Mesh read (s)	CSR pattern (s)	Vector alloc. (ms)	FEM assembly (s)	BC (s)	CG solver (s)	Compute total (s)	Pipeline total (s)
1	0.776	0.869	0.500	0.494	0.010	26.865	28.239	29.015
2	0.747	0.863	0.507	0.494	0.010	24.595	25.964	26.710
3	0.747	0.866	0.507	0.502	0.010	24.275	25.654	26.400
4	0.749	0.867	0.522	0.497	0.010	24.579	25.954	26.703
5	0.747	0.861	0.515	0.497	0.010	24.596	25.965	26.712

Correctness results

Run	CG iterations	Reported residual $\times 10^{-11}$	Recomputed residual $\times 10^{-10}$	Largest boundary $\lvert x_i \rvert$	Largest nodal error $\times 10^{-7}$	CG test
1	1280	9.841	1.402	0.000	3.631	Passed
2	1280	9.841	1.402	0.000	3.631	Passed
3	1280	9.841	1.402	0.000	3.631	Passed
4	1280	9.841	1.402	0.000	3.631	Passed
5	1280	9.841	1.402	0.000	3.631	Passed

Table 7: Raw performance and correctness results for the serially optimized build.

Performance timings

Run	Mesh read (s)	CSR pattern (s)	Vector alloc. (ms)	FEM assembly (s)	BC (s)	CG solver (s)	Compute total (s)	Pipeline total (s)
1	0.788	0.321	0.125	0.516	0.008	21.587	22.433	23.221
2	0.734	0.320	0.126	0.515	0.008	23.825	24.668	25.403
3	0.745	0.323	0.154	0.518	0.008	22.533	23.382	24.127
4	0.778	0.321	0.125	0.515	0.008	23.564	24.409	25.187
5	0.756	0.322	0.126	0.509	0.008	23.416	24.255	25.011

Correctness results

Run	CG iterations	Reported residual $\times 10^{-11}$	Recomputed residual $\times 10^{-10}$	Largest boundary $\lvert x_i \rvert$	Largest nodal error $\times 10^{-7}$	CG test
1	1279	9.995	1.409	0.000	3.631	Passed
2	1279	9.995	1.409	0.000	3.631	Passed
3	1279	9.995	1.409	0.000	3.631	Passed
4	1279	9.995	1.409	0.000	3.631	Passed
5	1279	9.995	1.409	0.000	3.631	Passed